FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA
"JESÚS OVIEDO PÉREZ"

TALLER DE LABORATORIO N.º 4

Monitoreo con APM y prueba de estrés sobre el CRUD

Presentado por:
David Felipe Garcia Ortiz
Juan David Claros Cachaya

Presentado a:
Ing. Juan Carlos Polania Cortes

Optativa IV – Sistemas de Tiempo Real Distribuidos
Ingeniería de Software – Semestre IX
Periodo 2026-2
Neiva, octubre de 2026

Contenido

1. Descripción de la solución
2. Entorno dockerizado y prueba de aceptación
3. Sistema APM e instrumentación
4. Diseño de la prueba de estrés
5. Resultados por escalón
6. Determinación del límite de la infraestructura
7. Preguntas de análisis
8. Anomalías observadas y limitaciones
9. Entregables
10. Declaración de uso de herramientas de inteligencia artificial

1. Descripción de la solución

Se partió del proyecto del **Laboratorio N.º 3**: un CRUD de pagos con una API en FastAPI, MySQL, RabbitMQ y un consumidor que procesa los pagos de forma asíncrona, todo en Docker. Sobre ese proyecto se hicieron tres cambios:

1. Se agregó la operación que faltaba para cumplir el punto 4 del taller: **actualizar** (`PUT /pagos/{id}`). Así la API expone crear, listar, consultar y actualizar.
2. Se instrumentó la aplicación con **OpenTelemetry** y se desplegó un **APM** completo en contenedores: OpenTelemetry Collector, Jaeger, Prometheus y Grafana.
3. Se construyó una **prueba de estrés con k6**, también en contenedor, con el escenario escrito en `carga/escenario.js` y un análisis automático que produce la tabla del punto 8 y las gráficas del punto 9.

Tabla 1. Operaciones del CRUD

Operación	Método y ruta	Qué hace en la base de datos
Crear	`POST /pagos`	`INSERT` + `COMMIT` y publica el id del pago en la cola `pagos.registrados`
Listar	`GET /pagos`	`SELECT` de los últimos 100 pagos con su procesamiento (`LEFT JOIN`)
Consultar	`GET /pagos/{id}`	`SELECT` de un pago por su clave primaria
Actualizar (nueva)	`PUT /pagos/{id}`	`UPDATE` de referencia, valor y medio de pago + `COMMIT`; 404 si no existe

Tabla 2. Componentes del entorno (todos en contenedores)

Componente	Imagen	Responsabilidad	Límite
API	build `./api` (python:3.12-slim)	CRUD de pagos instrumentado (un proceso uvicorn)	1 CPU / 512 MB
MySQL	`mysql:8.4`	Base de datos, volumen `mysql_data`, `max_connections` = 151	1 CPU / 1 GB
RabbitMQ	`rabbitmq:4-management-alpine`	Cola `pagos.registrados`	0,5 CPU / 512 MB
Consumidor	build `./consumer`	Procesa cada pago (1 s) y lo marca PROCESADO	0,5 CPU / 256 MB
OTel Collector	`otel/opentelemetry-collector-contrib:0.111.0`	Recibe la telemetría; calcula métricas RED con spanmetrics	1 CPU / 512 MB
Jaeger	`jaegertracing/all-in-one:1.62.0`	Trazas individuales y pestaña Monitor	1 CPU / 1 GB
Prometheus	`prom/prometheus:v2.54.1`	Almacena todas las métricas	0,5 CPU / 512 MB
Grafana	`grafana/grafana:11.2.2`	Tablero del APM ya aprovisionado	0,5 CPU / 256 MB
mysqld-exporter	`prom/mysqld-exporter:v0.15.1`	Conexiones activas de MySQL	0,25 CPU / 64 MB
monitor-contenedores	build `./monitor-contenedores`	CPU y memoria de cada contenedor (como `docker stats`)	0,25 CPU / 128 MB
k6	`grafana/k6:0.54.0`	Generador de carga (perfil `prueba`)	2 CPU / 1 GB

2. Entorno dockerizado y prueba de aceptación

Los cuatro componentes que exige el taller (aplicación, base de datos, APM y generador de carga) corren en contenedores. En el equipo sólo se usan Docker, el navegador y el editor. La prueba de

aceptación es que, en un equipo que sólo tenga Docker, **un único comando** deja funcionando la aplicación, la base de datos y el APM:

```
docker compose up -d --build
```

El generador de carga y las herramientas de análisis y captura están en el perfil `prueba` de `docker compose`, así no arrancan con ese comando y sólo se ejecutan cuando se les invoca. El instructivo completo para levantar el entorno desde cero está en `README.md`.

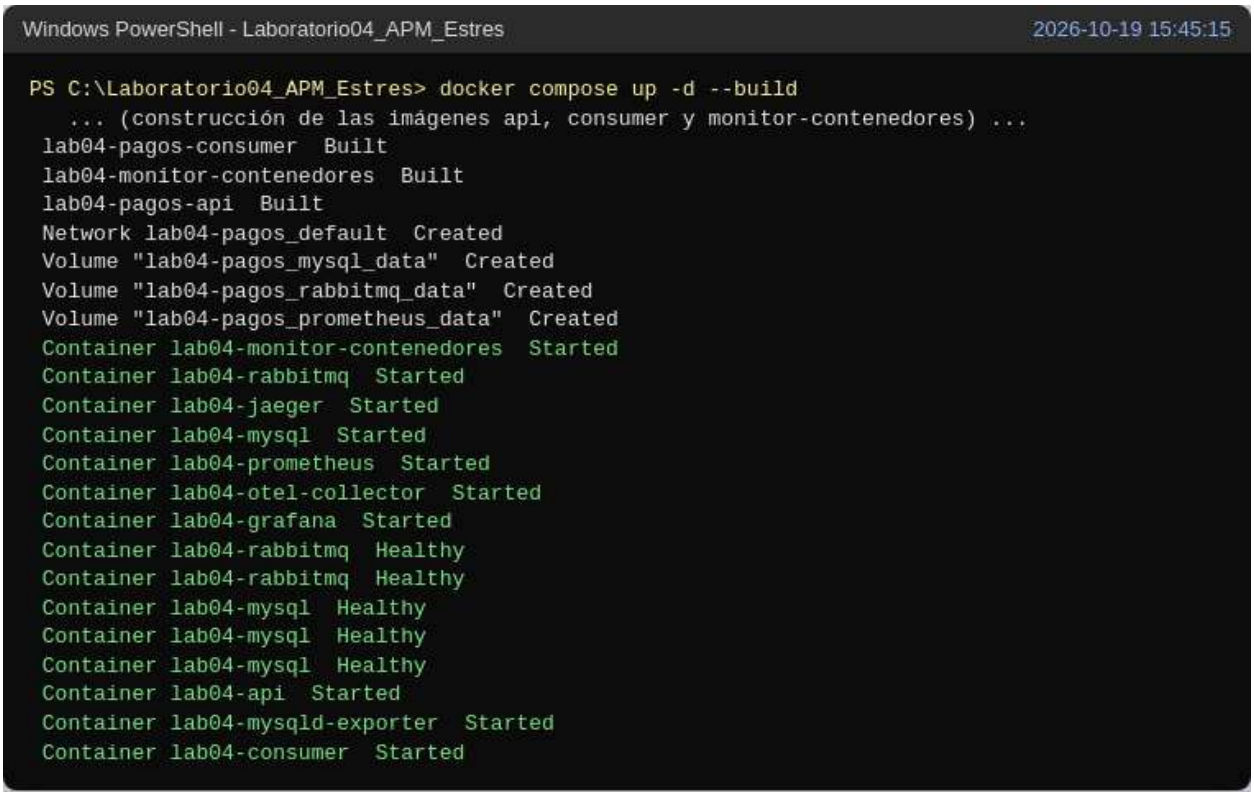


Figura 1. Levantamiento del entorno desde cero con un único comando (docker compose up -d --build). Se omite la parte intermedia de la construcción de imágenes.

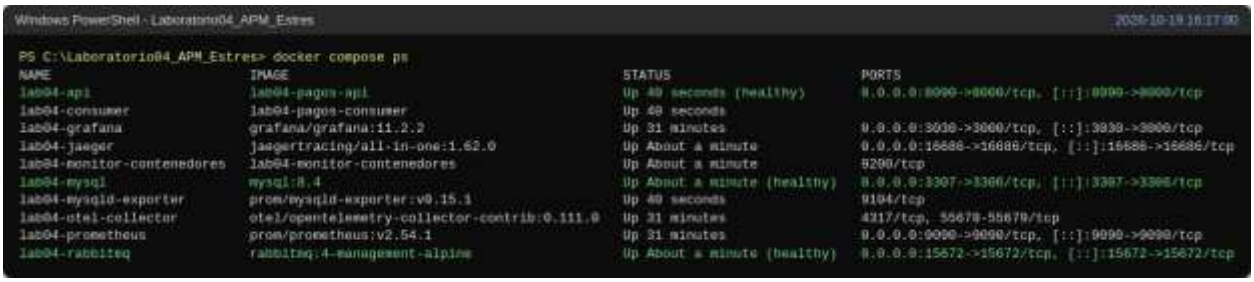


Figura 2. Todos los servicios en ejecución; la API, MySQL y RabbitMQ en estado healthy.

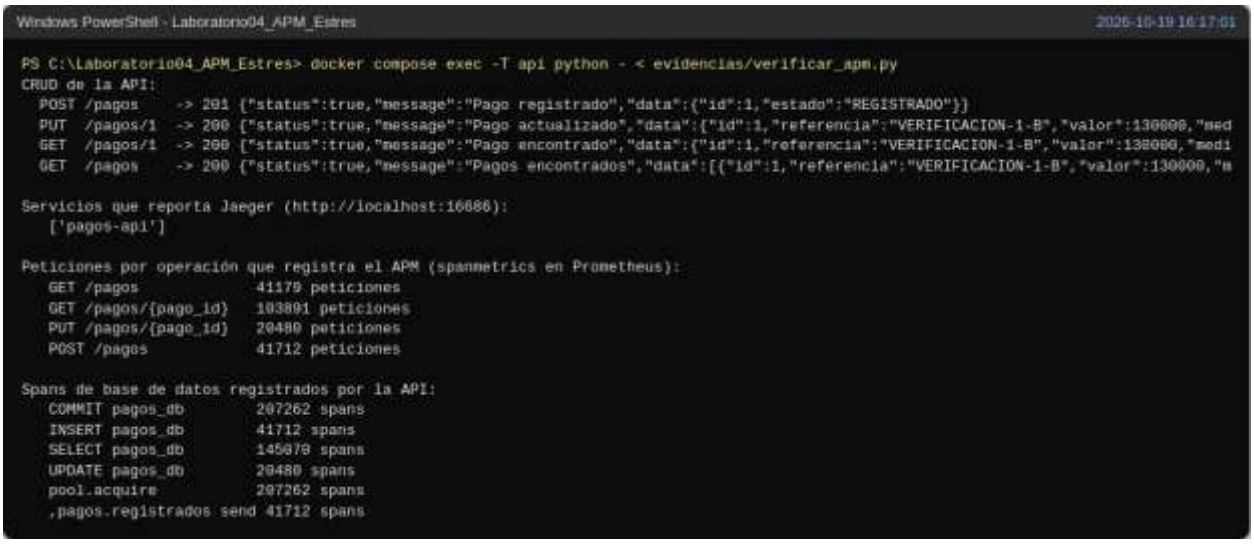


Figura 3. Verificación: las cuatro operaciones del CRUD responden y el APM ya registra sus peticiones y los spans de base de datos.

3. Sistema APM e instrumentación

3.1 Elección del APM

Se eligió **OpenTelemetry + Jaeger + Prometheus + Grafana**, que corresponde a dos de las opciones del punto 6 del taller: Jaeger + OpenTelemetry, complementado con Prometheus y Grafana para las métricas. Se descartó SigNoz porque necesita unos 4 GB de memoria (ClickHouse) y Docker Desktop en el equipo de medición sólo tiene 5,8 GB. Con esta composición el APM completo usa menos de 200 MB en reposo.

Requisito del punto 6	Cómo se observa
Peticiones por segundo por operación	Conector spanmetrics del Collector → Prometheus → panel de Grafana y pestaña Monitor de Jaeger
Latencia en percentiles p50, p95 y p99	Histograma de duración de spanmetrics (cubetas de 2 ms a 30 s; una cubeta exacta en 500 ms)
Tasa de error por operación	Spans de servidor con estado ERROR (5xx), por operación
Traza individual: tiempo en la aplicación y en la base de datos	Jaeger: cada petición muestra el span del endpoint y, dentro, la espera por conexión (`pool.acquire`), cada sentencia SQL y la publicación en RabbitMQ

3.2 Instrumentación de la aplicación

- **FastAPI (automática).** Un span `SERVER` por petición, con la ruta (`GET /pagos/{pago_id}`), el método y el código HTTP.
- **aio-pika (automática).** Un span `PRODUCER` por cada publicación en RabbitMQ, y propagación del contexto al consumidor.
- **MySQL (manual).** aiomysql no tiene instrumentación oficial. Por eso `app.py` abre un span `CLIENT` por cada sentencia (`SELECT`, `INSERT`, `UPDATE`, `COMMIT`), con la sentencia SQL como atributo.
- **Espera del pool (manual).** El span `pool.acquire` mide cuánto esperó la petición a que se liberara una conexión del pool. A la vez se publican las métricas `db.pool.en_uso`, `db.pool.maximo` y `db.pool.en_espera`.

- **Consumidor.** Se instrumenta sin tocar su código, con `opentelemetry-instrument` (pika y PyMySQL).

El Collector calcula las métricas sobre el **100 % de los spans**. A Jaeger envía el **20 %** de las trazas completas (muestreo por traza), suficiente para inspeccionarlas una a una sin que Jaeger se convierta en un cuello de botella. Durante las pruebas el Collector no rechazó ningún span y no falló ningún envío a Jaeger. Las métricas de CPU y memoria salen de un exportador propio (`monitor-contenedores`) que lee las mismas estadísticas que `docker stats`. Las conexiones de MySQL salen de `mysqld-exporter`.

```
# api/app.py: cada sentencia SQL queda en un span CLIENT
async def ejecutar(cur, sql, params=()):
    operacion = sql.split(None, 1)[0].upper()
    with tracer.start_as_current_span(f"{operacion} {DB_CONFIG['db']}", kind=SpanKind.CLIENT,
                                      attributes={**ATRIBUTOS_BD, "db.operation": operacion,
                                                  "db.statement": " ".join(sql.split())}):
        await cur.execute(sql, params)
```

3.3 Capturas del APM

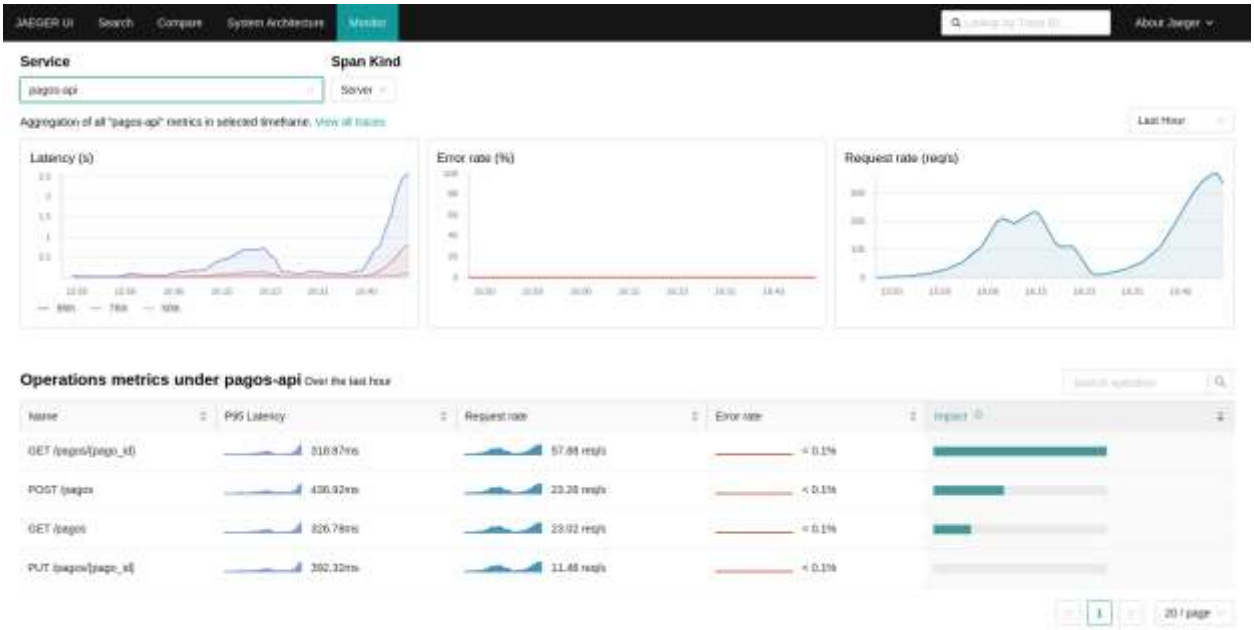


Figura 4. Panel del servicio pagos-api en Jaeger (pestaña Monitor): latencia, tasa de error y peticiones por segundo de cada operación.



Figura 5. Grafana: peticiones por segundo por operación del CRUD durante la corrida2.

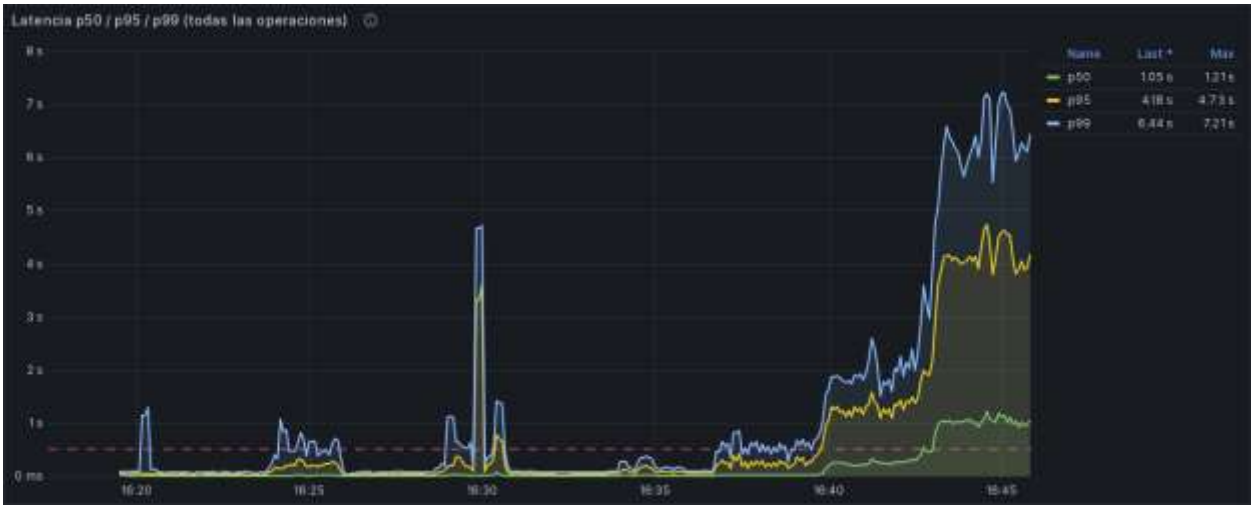


Figura 6. Grafana: latencia p50, p95 y p99. La línea roja es el criterio p95 ≤ 500 ms.

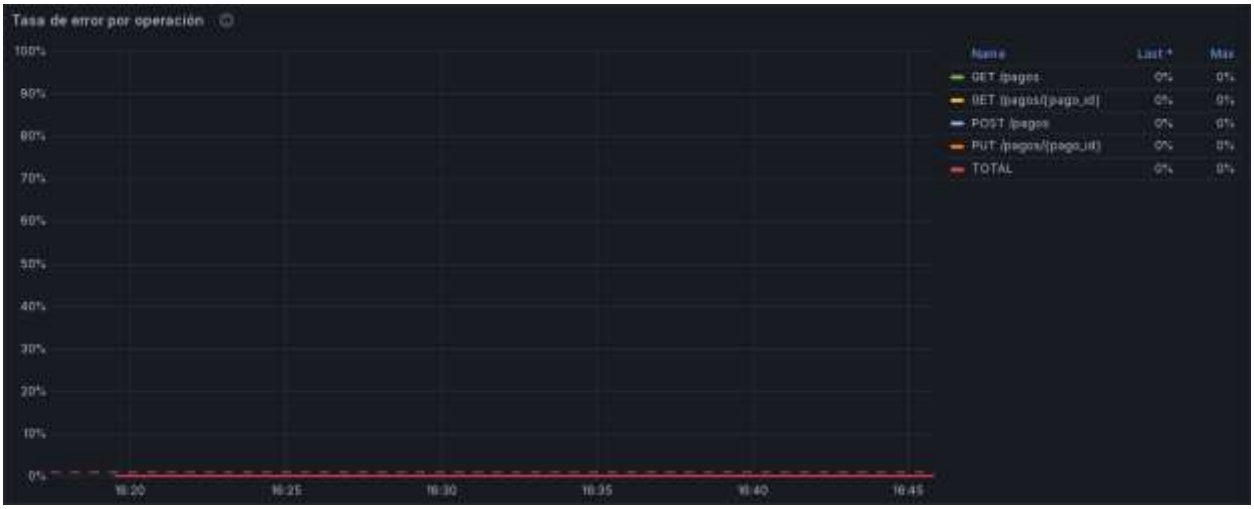


Figura 7. Grafana: tasa de error por operación (spans de servidor con estado ERROR).

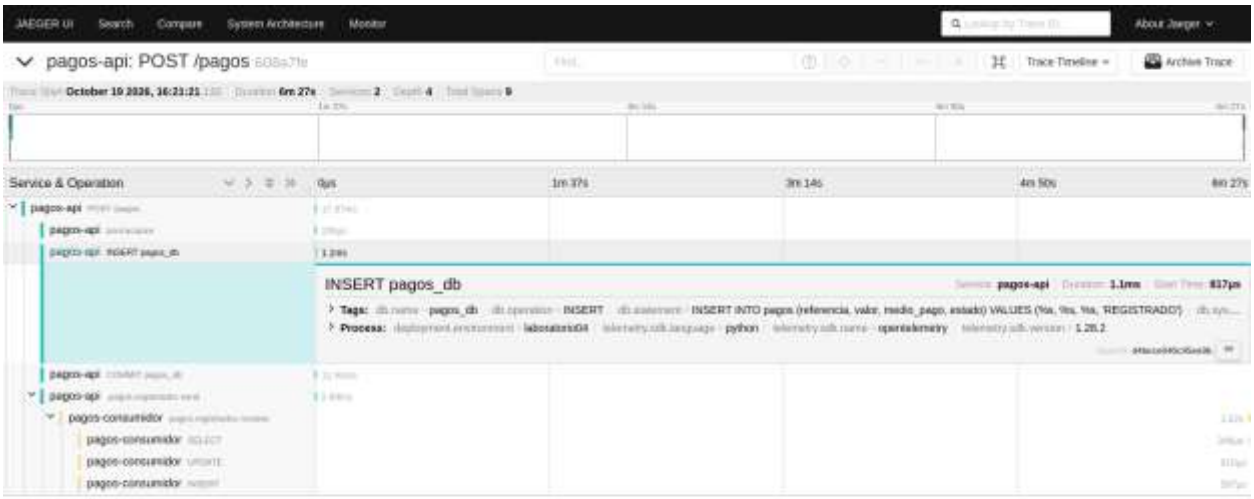


Figura 8. Traza individual de POST /pagos con 10 usuarios: espera de pool (µs), INSERT, COMMIT y publicación en RabbitMQ. El detalle del span INSERT muestra la sentencia SQL.

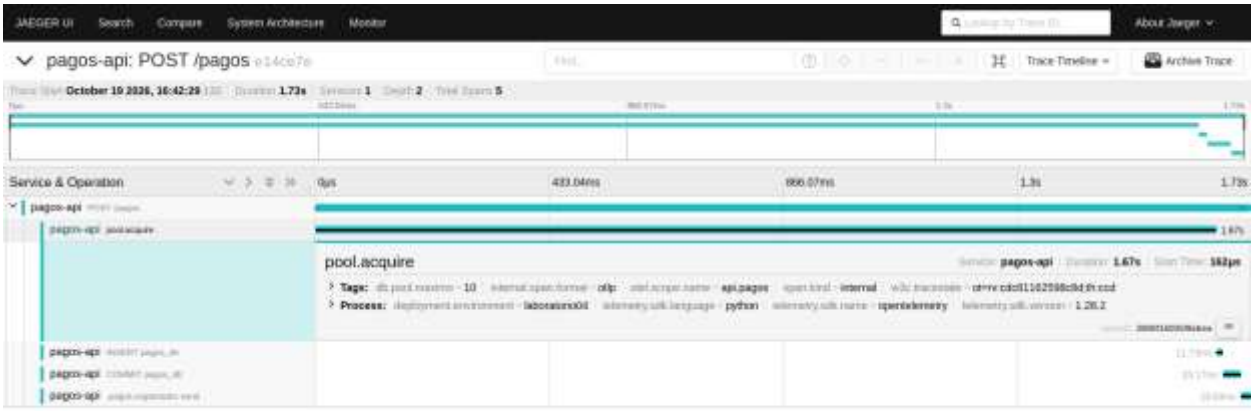


Figura 9. La misma operación con 600 usuarios: de 1,73 s, 1,67 s son espera por una conexión libre del pool (pool.acquire); MySQL sólo suma unos 45 ms.

4. Diseño de la prueba de estrés

4.1 Escenario (carga/escenario.js)

- **Operaciones y proporción declarada:** 50 % consultar un pago, 20 % listar, 20 % crear y 10 % actualizar. Es decir, **70 % lecturas y 30 % escrituras**.
- **Datos variables:** cada creación y actualización envía referencia, valor y medio de pago aleatorios. Las consultas y actualizaciones eligen al azar un id entre todos los pagos existentes. Antes de empezar se crean 300 pagos.
- **Usuario virtual:** después de cada petición espera entre 0,5 s y 1,5 s (1 s en promedio). Así N usuarios generan como máximo unas N peticiones por segundo y el número de usuarios concurrentes tiene sentido. Cada petición tiene un *timeout* de 10 s.
- **Escalones:** 1 minuto de calentamiento con 5 usuarios, que se descarta. Luego un escalón de **3 minutos** por cada nivel, con una rampa de 10 s entre escalones. De cada escalón se descartan los primeros 15 s y se analizan los 165 s restantes.
- **Progresión:** la declarada fue 10, 25, 50, 100, 200 y 400 usuarios. En la corrida1 el escalón de 400 todavía cumplía los criterios. Por eso, como quedó previsto en la declaración, se hizo la **corrida2** con 600 y 800 usuarios adicionales, **sin cambiar los criterios**.

4.2 Criterios de saturación (declarados antes de medir)

Los criterios quedaron escritos en `CRITERIOS_SATURACION.md` el **19 de octubre de 2026**, antes de la primera medición. Su hora de modificación, **11:52:54**, es anterior a la primera prueba de carga del día (una prueba de humo de 2 minutos para validar las herramientas, a las 12:08) y a la medición oficial (15:45). El archivo no se modificó después. Un escalón cumple si satisface los tres criterios a la vez; el punto de saturación es el último escalón que los cumple.

Criterio	Umbral declarado	Fuente
Tasa de error	≤ 1 %	APM (spans 5xx) y k6 (incluye *timeouts*); se toma el mayor
Latencia p95	≤ 500 ms	APM, todas las operaciones juntas
Crecimiento del throughput	Al menos +10 % respecto al escalón anterior	APM (peticiones/s atendidas)

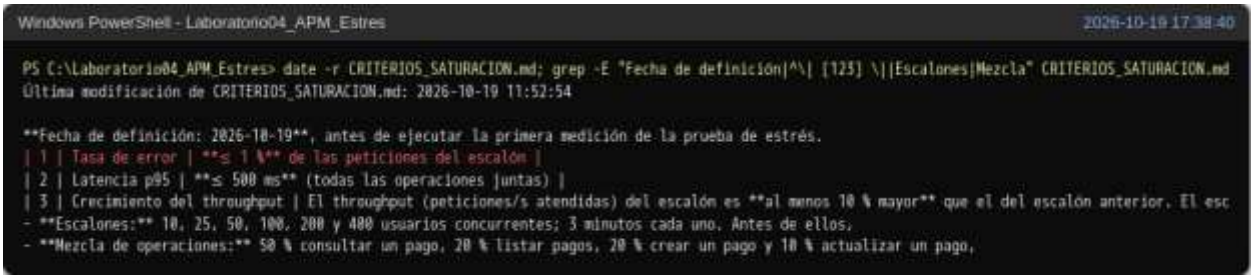


Figura 10. Hora de modificación de CRITERIOS_SATURACION.md (11:52:54 del 19-oct-2026) y sus líneas principales.

4.3 Ficha del equipo de medición

Elemento	Valor
Procesador	12th Gen Intel Core i5-12450H (8 núcleos: 4 de rendimiento y 4 de eficiencia; 12 hilos)
Memoria del equipo	16 GB
Sistema operativo	Windows 11 Pro 10.0.26300

Elemento	Valor
Docker	Docker Desktop, Docker Engine 28.3.3 sobre WSL2 (kernel 6.6.87.2)
Recursos para Docker	12 CPU lógicas y 5,8 GB de memoria
Límites de los contenedores	API 1 CPU / 512 MB; MySQL 1 CPU / 1 GB; RabbitMQ 0,5 / 512 MB; consumidor 0,5 / 256 MB; Collector 1 / 512 MB; Jaeger 1 / 1 GB; Prometheus 0,5 / 512 MB; Grafana 0,5 / 256 MB; k6 2 CPU / 1 GB
Configuración relevante	Un proceso uvicorn; pool de la API de 10 conexiones (`DB_POOL_MAX`); MySQL `max_connections` = 151

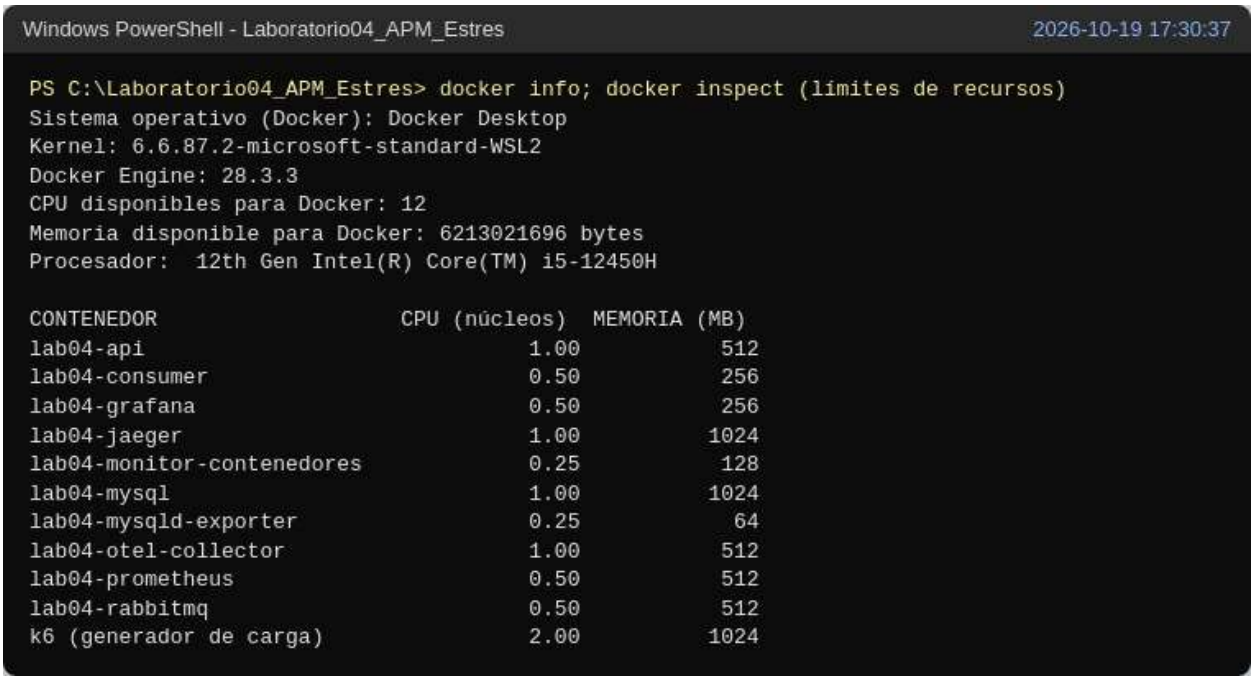


Figura 11. Ficha del equipo y límites de recursos asignados a cada contenedor, leídos de Docker.

5. Resultados por escalón

5.1 Línea base (entorno en reposo, con el APM corriendo)

Antes de cada corrida el entorno quedó 2 minutos en reposo con el APM encendido, como recomienda el taller. Así su costo queda incluido en la línea base. En reposo la API usa **48 MB** y menos del 1 % de CPU. MySQL usa **451 MB** (casi todo es su *buffer pool*) y mantiene 3 conexiones (la del exportador y las del pool mínimo). El APM completo (Collector, Jaeger, Prometheus y Grafana) suma unos **160 MB** (≈190 MB con los exportadores) y menos del 1 % de CPU.

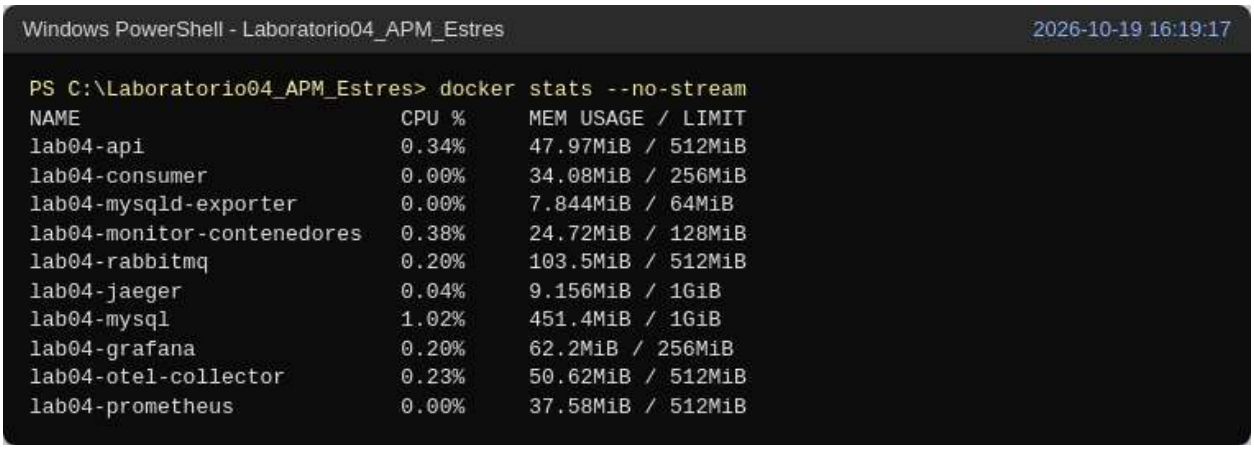


Figura 12. Línea base de la corrida2: consumo de cada contenedor en reposo (docker stats).

5.2 Corrida2: tabla de resultados (8 escalones)

Es la corrida principal: **8 escalones** de 10 a 800 usuarios, de 3 minutos cada uno, ejecutada el 19-oct-2026 entre las 16:20 y las 16:45. La latencia y los errores salen del APM. La CPU y la memoria salen de las estadísticas de los contenedores. Las conexiones son el máximo de `Threads_connected` de MySQL frente a `max_connections`, y el uso del pool de la API (en uso / tamaño y peticiones esperando conexión, en promedio).

Tabla 3. Corrida2: datos registrados en cada escalón (punto 8 del taller)

Esc.	Usuarios	Pet./s	p50 (ms)	p95 (ms)	p99 (ms)	Error %	CPU app	CPU BD	Memoria app / BD (MB)	Conex. BD (activas / máx.) y pool app	Cumple
1	10	9,8	4	32	70	0,00	7 %	5 %	50 / 460	5 / 151 pool 1/10; espera 0	Sí
2	25	23,9	7	162	413	0,00	26 %	16 %	52 / 477	12 / 151 pool 4/10; espera 0	Sí
3	50	47,3	7	141	693	0,00	31 %	12 %	52 / 481	12 / 151 pool 6/10; espera 0	Sí
4	100	98,7	4	38	76	0,00	32 %	11 %	53 / 485	13 / 151 pool 4/10; espera 0	Sí
5	200	194,6	8	85	197	0,00	56 %	19 %	58 / 488	12 / 151 pool 10/10; espera 2	Sí
6	400	377,7	18	247	568	0,00	83 %	23 %	67 / 492	12 / 151 pool 10/10; espera 44	Sí ← punto de saturación
7	600	426,6	252	1.343	1.985	0,00	99 %	25 %	77 / 496	12 / 151 pool 10/10; espera 202	No (p95)
8	800	357,8	1.041	4.225	6.527	0,03	100 %	28 %	88 / 501	12 / 151 pool 10/10; espera 484	No (p95, throughput)

La mezcla real coincidió con la declarada. En el escalón de 400 usuarios: consultar 190,7 pet./s (50 %), listar 75,0 (20 %), crear 74,9 (20 %) y actualizar 37,1 (10 %). El generador de carga nunca fue el cuello de botella: k6 usó como máximo el 37 % de una CPU, de las 2 que tenía asignadas.

5.3 Corrida1: los 6 escalones declarados

La corrida1 aplicó exactamente la progresión declarada (10 a 400 usuarios). Sus resultados casi coinciden con los de la corrida2 en los mismos escalones; en 400 usuarios dio 376,4 frente a 377,7

pet./s. Pero el último escalón todavía cumplía los tres criterios, así que el sistema no había saturado. Por eso se hizo la corrida2 con 600 y 800 usuarios adicionales, como estaba previsto en la declaración.

Tabla 4. Corrida1: datos registrados en cada escalón

Esc.	Usua-rios	Pet./s	p50 (ms)	p95 (ms)	p99 (ms)	Error %	CPU app	CPU BD	Memoria app / BD (MB)	Conex. BD (activas / máx.) y pool app	Cumple
1	10	9,9	4	26	90	0,00	6 %	3 %	50 / 448	5 / 151 pool 2/10; espera 0	Sí
2	25	24,7	4	26	79	0,00	10 %	6 %	51 / 458	8 / 151 pool 3/10; espera 0	Sí
3	50	47,8	7	175	397	0,00	31 %	20 %	53 / 469	13 / 151 pool 6/10; espera 0	Sí
4	100	99,0	4	34	74	0,00	30 %	11 %	54 / 480	13 / 151 pool 2/10; espera 0	Sí
5	200	190,6	5	47	90	0,00	52 %	16 %	61 / 484	12 / 151 pool 7/10; espera 0	Sí
6	400	376,4	17	244	527	0,00	89 %	26 %	71 / 489	12 / 151 pool 10/10; espera 17	Sí ← punto de saturación

5.4 Tablero del APM durante la corrida2

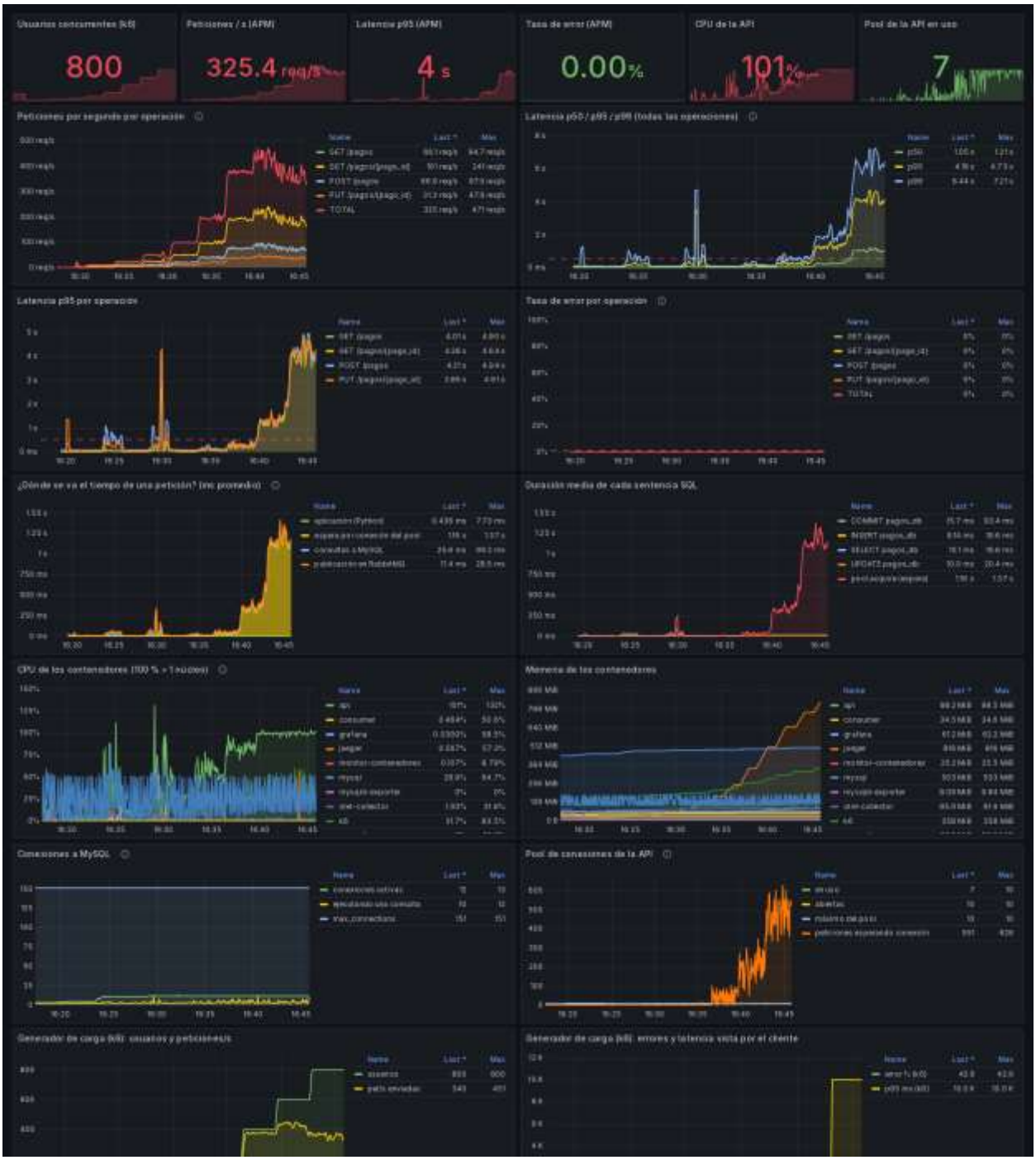


Figura 13. Tablero de Grafana durante toda la corrida2. Se ve cómo la latencia se dispara y el pool se llena cuando la CPU de la API llega al 100 %.

6. Determinación del límite de la infraestructura

6.1 Punto de saturación

Aplicando los criterios declarados a la corrida2, los seis primeros escalones (10 a 400 usuarios) los cumplen y **el escalón 7 (600 usuarios) es el primero que falla**. Allí la p95 sube a 1.343 ms, casi el

triple del umbral. Por lo tanto, **el punto de saturación es el escalón 6: 400 usuarios concurrentes y 377,7 peticiones por segundo**, con p95 de 247 ms y 0 % de error. Con 800 usuarios el throughput incluso cae (357,8 pet./s), la p95 llega a 4,2 s y aparecen *timeouts*.

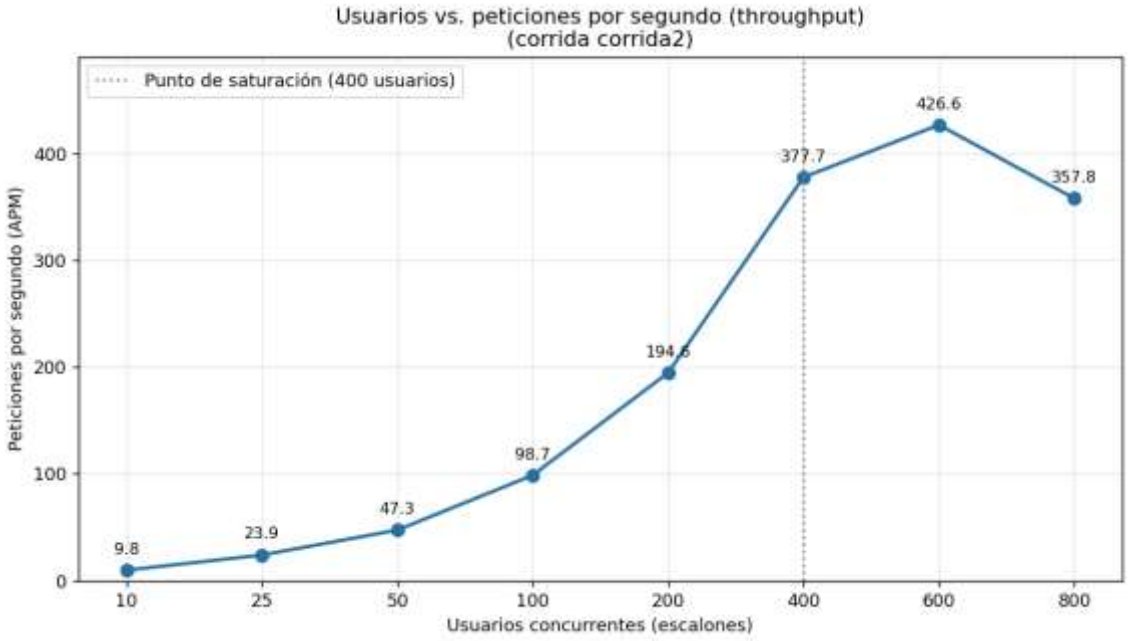


Figura 14. Usuarios frente a peticiones por segundo. El throughput crece casi en línea recta hasta 400 usuarios, se aplana en 600 (≈427 pet./s) y cae en 800.

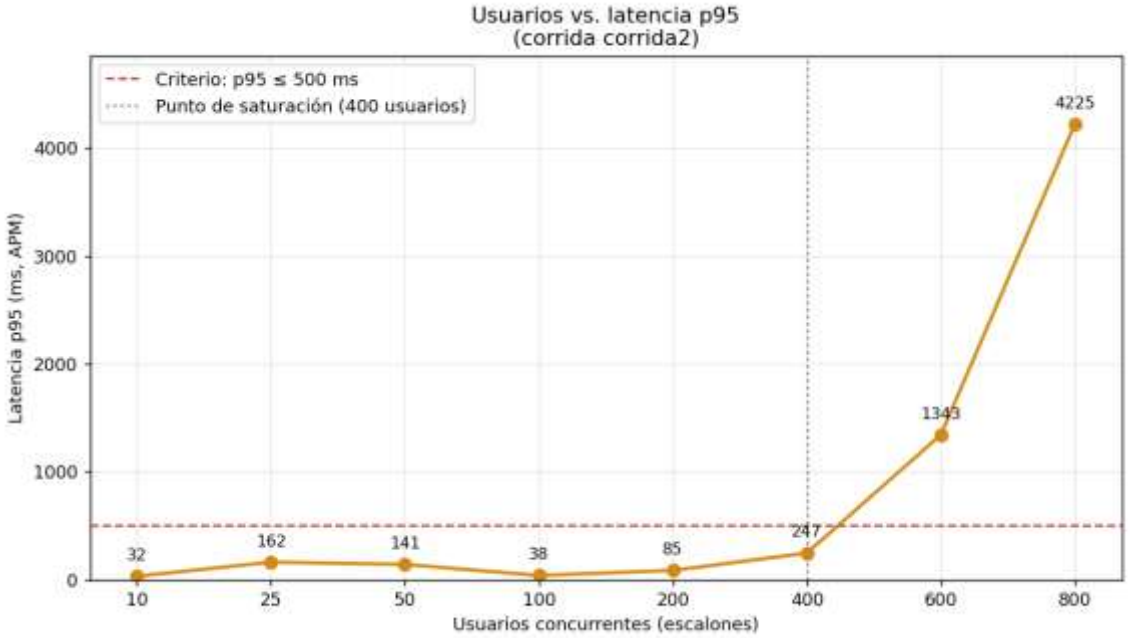


Figura 15. Usuarios frente a latencia p95. El criterio de 500 ms se rompe entre 400 y 600 usuarios.

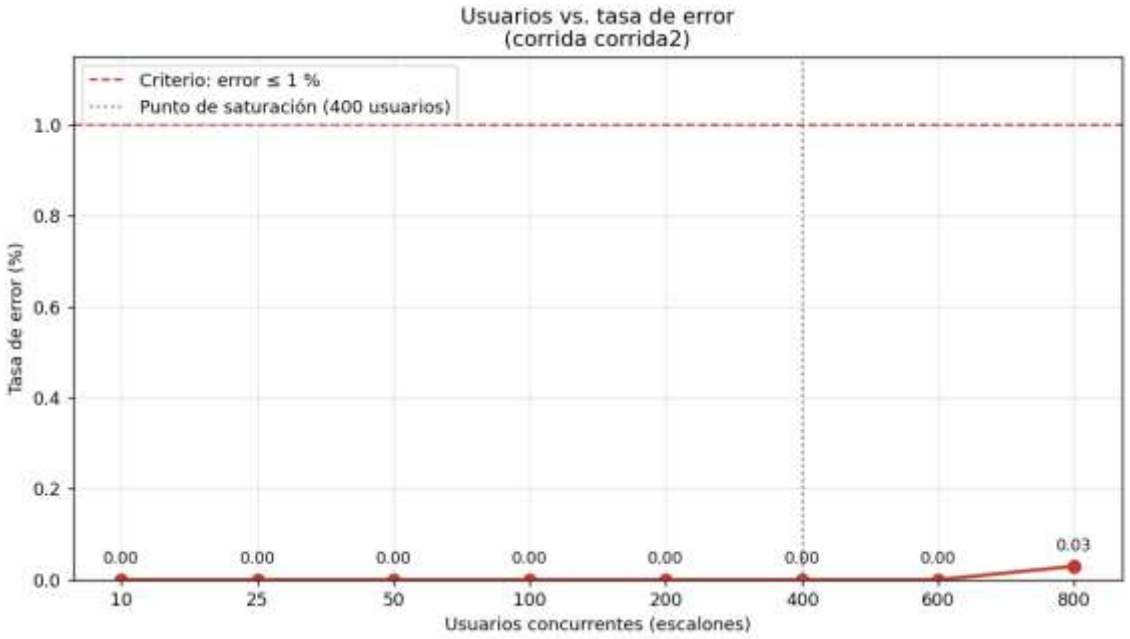


Figura 16. Usuarios frente a tasa de error. Se mantiene en 0 % hasta 600 usuarios; en 800 aparecen 0,03 % de timeouts (lado del cliente).

6.2 Recurso que impone el límite

Las tres fuentes de evidencia coinciden:

- **Estadísticas de los contenedores:** la CPU de la API pasa de 56 % (200 usuarios) a 83 % (400) y a 99 % (600), es decir, a su límite de 1 CPU. MySQL nunca pasa del 28 %. La memoria de la API no supera 88 MB de 512 y MySQL usa 12 de sus 151 conexiones.
- **APM:** el pool de 10 conexiones se llena desde 200 usuarios y, a partir de 400, las peticiones forman fila para obtener una conexión: 44 esperando en promedio con 400 usuarios y 202 con 600. La traza de la Figura 9 muestra el resultado: 1,67 s de 1,73 s en `pool.acquire`.
- **Experimento de control:** con el pool en **30 conexiones** (sección 7, pregunta 5) el throughput máximo **no subió** (405,5 pet./s frente a 426,6) y la CPU de la API siguió al 100 %. Si el pool fuera el límite, ampliarlo habría subido el techo.

Conclusión: **el recurso que impone el límite es la CPU de la API**, un único proceso de uvicorn con 1 núcleo asignado. Cada petición cuesta entre 2,2 y 2,9 ms de CPU, así que un núcleo alcanza para unas 350 a 450 pet./s, que es justo donde se aplana la curva (≈427 pet./s). El pool de conexiones es **lo primero que se llena** y el sitio donde se forma la fila, pero es la consecuencia, no la causa.

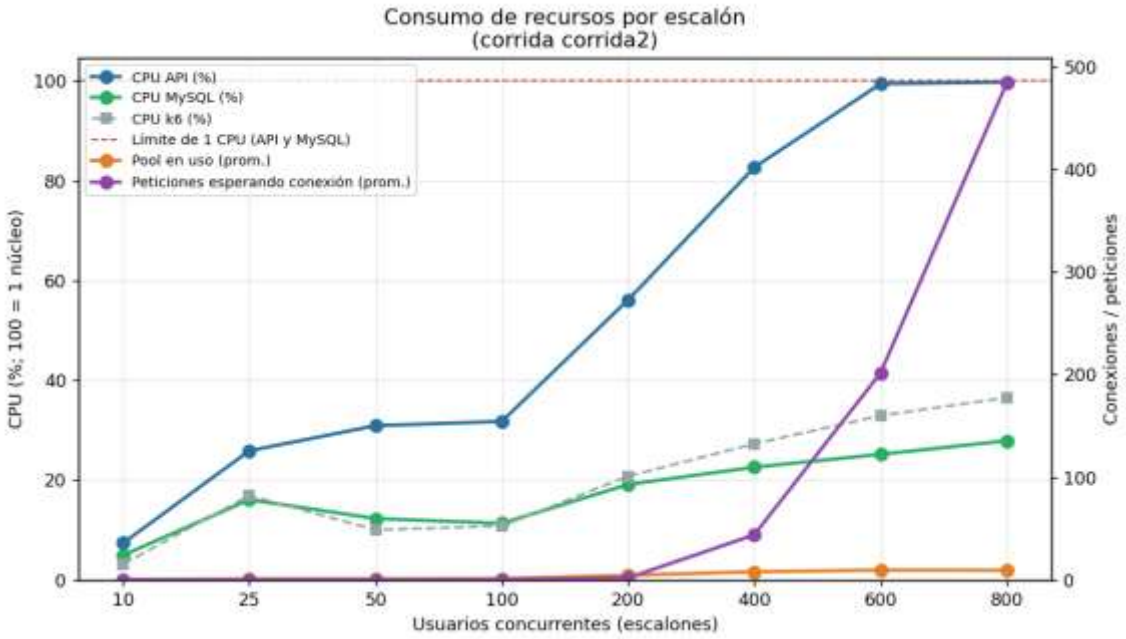


Figura 17. Consumo de recursos por escalón (corrida2): la CPU de la API llega al 100 % mientras MySQL se queda cerca del 25 % y las peticiones esperando conexión se disparan.

6.3 Repetición del escalón de saturación

Como recomienda el taller, se repitió el escalón de saturación (400 usuarios) y el siguiente. También se hizo una corrida de confirmación partiendo de datos limpios. El resultado **no se sostuvo con el mismo margen**:

Tabla 5. Repeticiones del escalón de saturación

Corrida	Condición	Usuarios	Pet./s	p95 (ms)	Error %	CPU API	ms de CPU por petición	Cumple p95 y error
corrida1	Equipo descansado; datos limpios	400	376,4	244	0,00	89 %	2,37	Sí
corrida2	Equipo descansado; datos limpios	400	377,7	247	0,00	83 %	2,19	Sí
repetición 1	Justo después de la corrida1, con su cola acumulada	400	344,5	693	0,00	96 %	2,78	No
repetición 2	Justo después de la corrida2 (≈81 600 mensajes en cola al terminar)	400	316,8	873	0,00	96 %	3,04	No
pool 30	Datos limpios, pool de 30 conexiones (≈1 h 35 min de carga)	400	350,7	627	0,00	94 %	2,68	No
confirmación	Datos limpios, cola vacía (≈1 h 20 min de carga continua)	400	157,7	7.239	2,97	105 %	6,65	No
confirmación	Mismo momento, 200 usuarios	200	185,4	312	0,00	74 %	3,97	Sí

La última columna es la que explica la diferencia. Con 400 usuarios, el mismo código gastó **2,2 a 2,4 ms de CPU por petición** con el equipo descansado y **2,7 a 3,0 ms** después de una hora de carga continua. En la confirmación llegó a 4,0 ms con sólo 200 usuarios. Ese encarecimiento afectó por igual a la API, a MySQL y a RabbitMQ, así que no es un efecto de la aplicación: el portátil perdió rendimiento bajo carga sostenida (ver sección 8). Como con 1 CPU el techo es CPU / costo por petición, un costo 25 % mayor baja el techo en la misma proporción y deja los 400 usuarios por encima del umbral de p95.

6.4 Respuesta a la pregunta central

La infraestructura soporta 377,7 peticiones por segundo con 400 usuarios concurrentes antes de incumplir los criterios declarados (p95 247 ms, error 0 %). Ese punto se midió en dos corridas independientes: 376,4 y 377,7 pet./s. El máximo absoluto es ≈427 pet./s con 600 usuarios, pero ya con p95 de 1,3 s. **El límite lo impone la CPU de la API** (un proceso de uvicorn con 1 núcleo).

El margen es estrecho. Con el equipo bajo carga sostenida, el mismo escalón de 400 usuarios dio una p95 de 627 a 873 ms. El límite que **se cumplió en todas las mediciones**, también en las del equipo degradado, es de **200 usuarios concurrentes y ≈185 a 195 peticiones por segundo**. Ese es el valor que recomendamos usar como capacidad garantizada de esta infraestructura.

7. Preguntas de análisis

7.1 ¿Qué se degradó primero al cruzar el punto de saturación: la latencia o la tasa de error? ¿Qué indica ese orden?

Primero se degradó la latencia. Al pasar de 400 a 600 usuarios la p95 subió de 247 ms a 1.343 ms y la p50 de 18 a 252 ms, mientras la tasa de error se mantuvo en 0 %. Los errores sólo aparecieron con 800 usuarios (0,03 %, 17 *timeouts* de 10 s de unas 291 000 peticiones). Además los vio el cliente (k6), no el servidor: el APM no registró ninguna respuesta 5xx en toda la prueba.

Ese orden indica que el sistema **no rechaza trabajo: lo pone en fila**. La espera por una conexión del pool no tiene tope ni tiempo máximo, así que cuando llega más trabajo del que la CPU puede atender las peticiones esperan cada vez más hasta que el cliente se rinde. Para un usuario real esto significa que la lentitud avisa mucho antes que los errores. También significa que el umbral de latencia es el criterio que realmente protege la experiencia. En producción convendría acotar esa fila (un *timeout* al pedir conexión, o limitar las peticiones concurrentes) para fallar rápido en vez de acumular segundos de espera.

7.2 Según las trazas del APM, ¿qué proporción del tiempo se consume en la aplicación y qué proporción en la base de datos? ¿Cambió al aumentar la carga?

Tabla 6. Corrida2: tiempo promedio de una petición según las trazas del APM (ms)

Usuarios	Total (ms)	Aplicación	Espera de pool	Consultas MySQL	Publicar en RabbitMQ	% BD + pool
10	9,1	1,4	0,0	5,5	2,2	61 %
25	34,2	2,4	0,5	22,3	9,0	67 %
50	49,4	2,1	16,5	23,3	7,5	81 %
100	9,7	1,2	0,1	6,4	2,0	67 %
200	20,5	1,2	4,0	11,6	3,7	76 %
400	53,0	0,9	32,0	14,6	5,5	88 %
600	378,4	0,8	347,7	20,7	9,1	97 %
800	1.198,7	1,2	1.161,3	24,7	11,5	99 %

Sí cambió, y mucho. Con poca carga (10 usuarios) una petición tarda 9,1 ms. De ellos, 5,5 ms (≈61 %) son consultas a MySQL, 2,2 ms la publicación en RabbitMQ y sólo 1,4 ms (≈15 %) código de la aplicación. La sentencia más cara es el `COMMIT` (4,2 ms en promedio con 10 usuarios, frente a 1,5 ms de un `SELECT`, porque MySQL escribe el log en disco), que la API ejecuta incluso después de las

lecturas. Con 400 usuarios la petición sube a 53,0 ms, de los cuales 32,0 son espera por conexión. Con 600 usuarios, 347,7 de 378,4 ms (92 %) son espera por una conexión del pool.

Hay que leer esto con cuidado. La API es asíncrona y corre en un solo proceso, así que cuando su CPU está al 100 % todo lo que espera (una conexión, la respuesta de MySQL, la confirmación de RabbitMQ) tarda más en ser atendido. Por eso el tiempo se acumula en los spans de espera aunque MySQL esté al 25 % de CPU. El experimento con pool de 30 lo confirma: la espera de pool bajó, pero el tiempo "en MySQL" pasó de 21 a 56 ms y la publicación de 9 a 237 ms. **La traza muestra dónde espera la petición, no necesariamente quién es lento.**

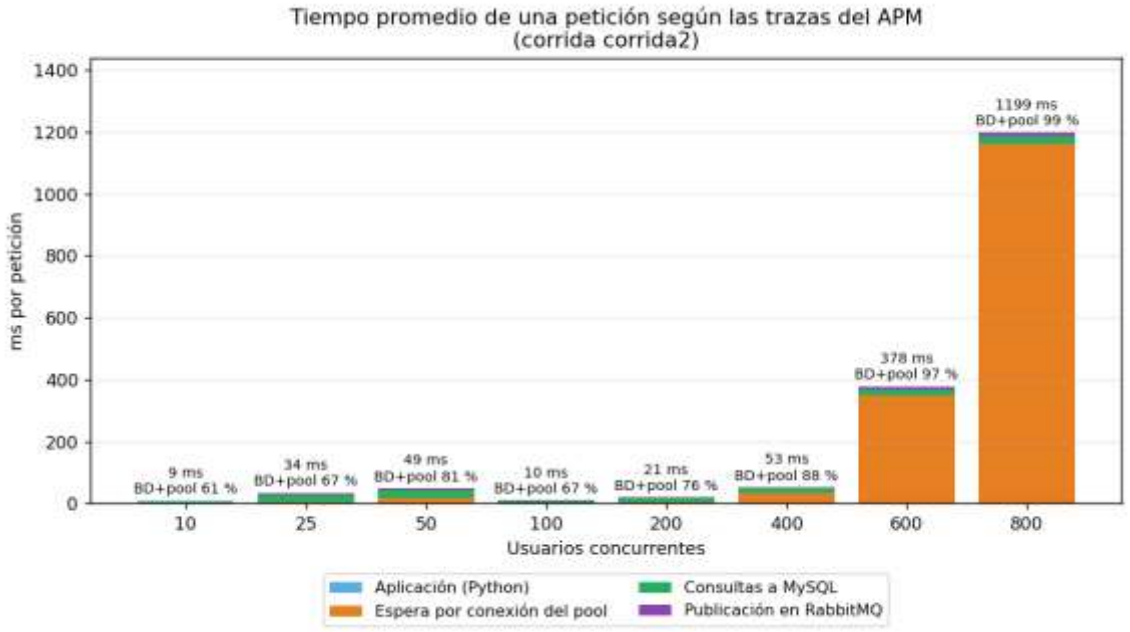


Figura 18. Desglose del tiempo promedio de una petición por escalón, según las trazas del APM.

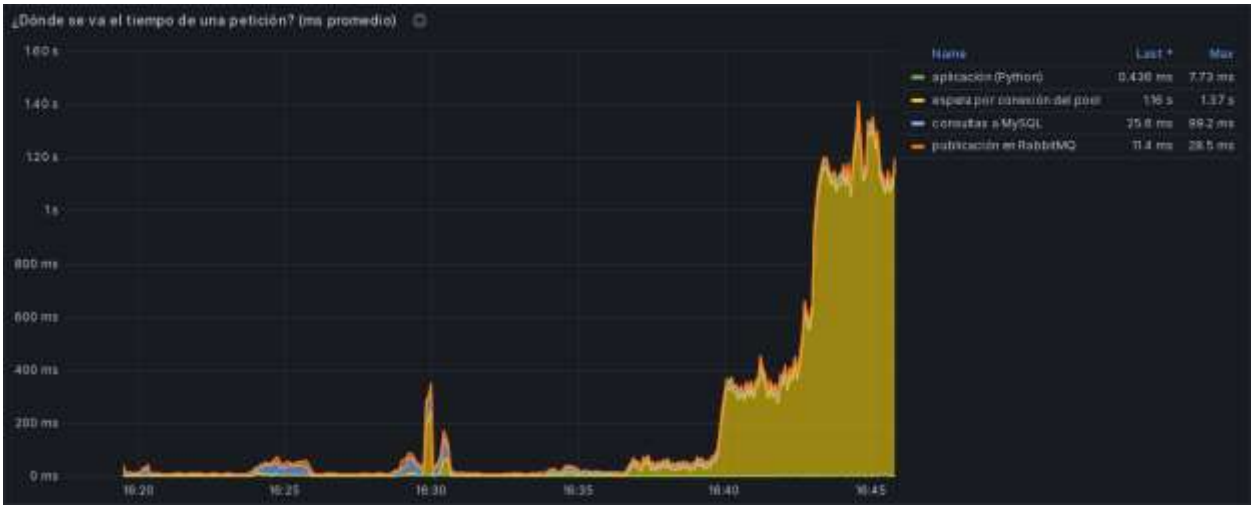


Figura 19. El mismo desglose en el tablero de Grafana durante la corrida2.

7.3 ¿El throughput siguió creciendo con cada escalón o se aplanó? Si se aplanó, ¿en qué valor?

Creció casi proporcionalmente a los usuarios hasta 400: entre +94 % y +144 % por escalón, con una eficiencia de 0,94 a 0,99 pet./s por usuario. **Se aplanó en ≈427 peticiones por segundo**: con 600 usuarios sólo creció 12,9 % aunque los usuarios aumentaron 50 %. Con 800 usuarios **cayó** a 357,8 pet./s (-16,1 %): con tantas peticiones en fila, el proceso gasta CPU en administrarlas y en *timeouts* en vez de completar trabajo. En las repeticiones con el equipo degradado el techo bajó a ≈340 a 405 pet./s.

7.4 ¿Qué se agotó primero: el procesador, la memoria o el conjunto de conexiones a la base de datos?

El conjunto de conexiones de la API fue lo primero en agotarse, pero el recurso que impone el límite es el procesador de la API.

- **Pool de conexiones:** con 200 usuarios ya llegó a 10 de 10 conexiones en uso, con 2 peticiones esperando en promedio, cuando la CPU de la API estaba al 56 %.
- **Procesador:** llegó al 83 % con 400 usuarios y al 100 % (su límite de 1 CPU) con 600. Desde ese punto el throughput dejó de crecer.
- **Memoria:** nunca fue problema. La API usó como máximo 88 MB de 512 y MySQL ≈500 MB de 1 GB.
- **Conexiones de MySQL:** 12 de 151 (`max_connections`); el límite de la base de datos quedó muy lejos.

El pool de 10 se llena primero porque es una fila, y una fila se llena en cuanto llega un poco más de trabajo del que se atiende. Que no sea la causa lo demuestra el experimento de la sección 7.5: con el triple de conexiones el throughput no aumentó y la CPU siguió al 100 %.



Figura 20. Pool de conexiones de la API durante la corrida2: 10 de 10 en uso y cientos de peticiones esperando conexión en los últimos escalones.

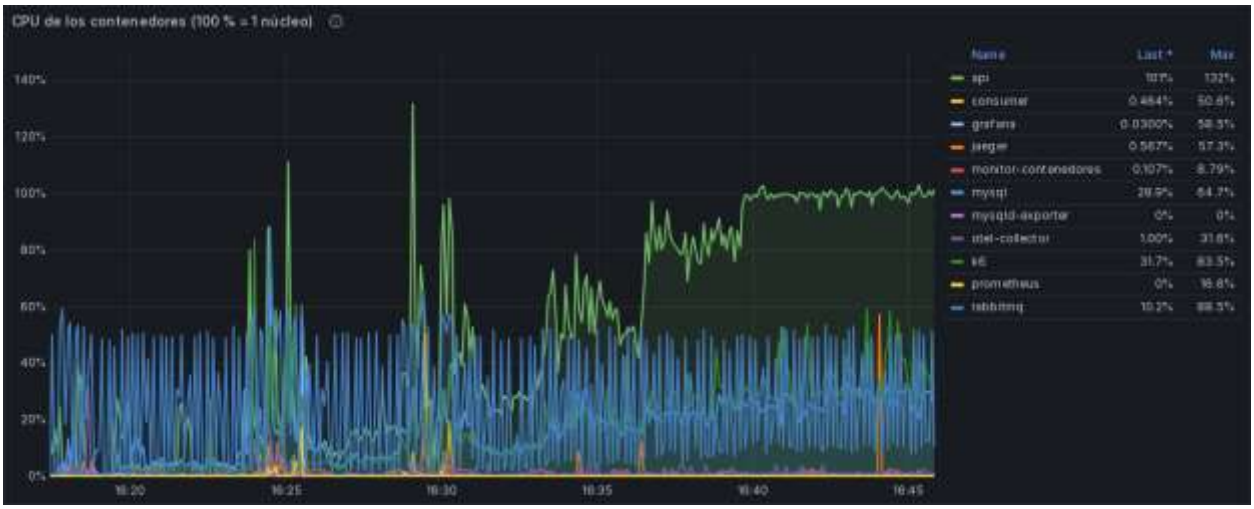


Figura 21. CPU de los contenedores durante la corrida2 (100 % = un núcleo). La API llega a su límite; MySQL no.

7.5 Proponga un cambio concreto para elevar el límite y explique, con base en sus mediciones, por qué cree que funcionaría

Primero se descartó el cambio "obvio", **agrandar el pool de conexiones**. Se probó con 30 conexiones (`DB_POOL_MAX=30`), en los mismos escalones de 400 y 600 usuarios:

Configuración	Usua-rios	Pet./s	p95 (ms)	CPU API	Espera pool (ms)	MySQL (ms)	RabbitMQ (ms)	Conex. MySQL
Pool 10 (corrida2)	400	377,7	247	83 %	32,0	14,6	5,5	12
Pool 10 (corrida2)	600	426,6	1.343	99 %	347,7	20,7	9,1	12
Pool 30	400	350,7	627	94 %	52,9	33,0	31,4	32
Pool 30	600	405,5	1.971	100 %	147,6	55,9	236,8	32

Con el triple de conexiones el throughput no subió y la CPU de la API siguió al 100 %. La espera sólo se movió del pool a MySQL y a RabbitMQ. Agrandar el pool no eleva el límite.

Cambio propuesto: ejecutar la API con 2 procesos de uvicorn y asignarle 2 CPU al contenedor, manteniendo un pool de 10 conexiones por proceso. En concreto: `--workers 2` en el comando de uvicorn y `cpus: 2.0` en `docker-compose.yml`.

Por qué creemos que funcionaría, con base en lo medido:

1. **El límite es la CPU de un solo proceso.** Cada petición cuesta ≈2,2 a 2,9 ms de CPU, y un núcleo da para ≈350 a 450 pet./s, justo el techo medido (≈427). Python ejecuta el código de un proceso en un solo núcleo, así que darle más CPU al contenedor sin más procesos no serviría. Dos procesos pueden usar dos núcleos y duplicar ese techo teórico.
2. **El resto de la infraestructura tiene margen.** Con 400 pet./s MySQL usó ≈25 % de su CPU y 12 de 151 conexiones. Con el doble de tráfico estaría en ≈50 % y con 2 × 10 + 3 ≈ 23 conexiones. RabbitMQ estuvo en ≈25 % y k6 en 37 % de una CPU.
3. **El pool no estorba.** El experimento mostró que con la CPU como cuello de botella más conexiones no ayudan. Con dos procesos, 10 conexiones por proceso (20 en total) siguen siendo suficientes y seguras frente a `max_connections` = 151.

Estimamos que el límite subiría a unas **700 a 800 pet./s y 700 a 800 usuarios** con la misma p95 de 500 ms, hasta que el siguiente recurso (MySQL o el disco de los `COMMIT`) empiece a pesar. Hay dos mejoras baratas que se suman, porque reducen la CPU por petición: quitar el `COMMIT` después de las

lecturas (con autocommit, una ida y vuelta menos a MySQL por consulta) e instalar `uvicorn[standard]` (bucle de eventos uvloop y parser httptools). Las dos medidas son verificables repitiendo los escalones 400 a 800 con el mismo escenario.

8. Anomalías observadas y limitaciones

- **Picos de latencia con poca carga.** En las dos corridas, los escalones de 25 y 50 usuarios tuvieron p95 de 141 a 175 ms, peor que con 100 usuarios (34 a 38 ms). En esos minutos la CPU de **todos** los contenedores subió a la vez sin que subiera la carga (a las 16:24, API 65 %, MySQL 65 %, RabbitMQ 88 % y k6 83 %). Eso apunta a una interferencia del equipo y no de la aplicación. La hipótesis más probable es la gestión de energía del portátil: el i5-12450H tiene núcleos de rendimiento y de eficiencia, y el plan de energía estaba en "Equilibrado". Los criterios se cumplieron igualmente.
- **Pérdida de rendimiento bajo carga sostenida.** Como muestra la tabla de repeticiones (sección 6.3), después de una hora de carga continua cada petición costó entre 20 % y 40 % más CPU, y hasta el triple en el peor momento (confirmación con 400 usuarios). La degradación también apareció partiendo de datos limpios y con la cola vacía, así que no la causó el estado acumulado sino el equipo. Por eso el informe da dos cifras: el punto de saturación con el equipo descansado y el límite que se sostuvo en todas las mediciones.
- **El costo del APM.** Medido en la línea base: ≈160 MB y menos del 1 % de CPU en reposo. Durante la corrida2 el Collector no rechazó spans ni falló envíos; en promedio usó 1,5 % de una CPU y Jaeger 0,8 % (con picos de 24 % y 57 % durante un segundo). Pero la instrumentación también corre dentro de la API: crear y exportar unos 6 spans por petición forma parte de los 2,2 ms de CPU por petición. Medir sin APM daría un techo algo mayor.
- **La cola de RabbitMQ crece durante la prueba.** El consumidor procesa un pago por segundo y la prueba crea ≈80 pagos por segundo, así que al terminar la corrida2 y su repetición había 81 652 mensajes pendientes. Es el comportamiento esperado del diseño asíncrono del Lab. 3. Antes de la confirmación y del experimento se reinició la cola.
- **Un intento del experimento se descartó.** En el primer intento con pool de 30, `docker compose run k6` recreó la API con la configuración por defecto (pool de 10) justo antes de la prueba. Se detectó porque el APM mostraba 10 conexiones y MySQL 12. Se corrigió exportando `DB_POOL_MAX` para todo el script (`evidencias/experimento_pool.sh`) y se repitió.
- **Escalones de baja carga con poco tráfico.** Con 10 usuarios cada escalón tiene ≈1 600 peticiones, así que su p99 es menos estable que la de los escalones altos.

9. Entregables

Entregable	Ubicación
Código fuente de la aplicación instrumentada	`api/app.py`, `api/telemetria.py`, `consumer/consumer.py`
Archivo del escenario de carga	`carga/escenario.js`
Archivos de definición de imágenes	`api/Dockerfile`, `consumer/Dockerfile`, `monitor-contenedores/Dockerfile`, `analisis/Dockerfile`, `evidencias/herramientas_captura/Dockerfile`
Archivo de composición del entorno	`docker-compose.yml`
Configuración del APM	`observabilidad/` (Collector, Prometheus, Grafana y tablero JSON)
Instructivo para levantar todo desde cero	`README.md`

Entregable	Ubicación
Declaración de los criterios de saturación (con fecha)	`CRITERIOS_SATURACION.md` (19-oct-2026) y sección 4.2
Tabla de resultados completa	Secciones 5.2 y 5.3; `resultados/<corrida>/tabla.md` y `tabla.csv`
Las tres gráficas del punto 9	Sección 6.1; `resultados/corrida2/grafica_1…3_*.png`
Capturas del APM	Secciones 3.3, 5.4 y 7; `evidencias/capturas/` y `evidencias/capturas_corrida1/`
Ficha del equipo	Sección 4.3
Respuestas a las cinco preguntas de análisis	Sección 7
Declaración de uso de herramientas de IA	Sección 10
Scripts para reproducir las mediciones	`evidencias/generar_evidencias.sh`, `corrida_ampliada.sh`, `confirmacion_limpia.sh`, `experimento_pool.sh`

10. Declaración de uso de herramientas de inteligencia artificial

En este taller se utilizó **Claude** (modelo Claude Opus 5.5, de Anthropic), a través de la herramienta **Claude Code**, como asistente de programación y de redacción. Se usó para:

- Interpretar el enunciado del taller y proponer la arquitectura del APM (OpenTelemetry, Jaeger, Prometheus y Grafana).
- Escribir y depurar el código: el endpoint `PUT /pagos/{id}`, la instrumentación con OpenTelemetry, el archivo de composición, el escenario de k6, el exportador de estadísticas de contenedores, el script de análisis y los scripts de captura de evidencias.
- Redactar la declaración de criterios con los valores sugeridos por el docente (error ≤ 1 %, p95 ≤ 500 ms y throughput creciente, cuantificado en +10 %), antes de ejecutar la primera medición.
- Ejecutar las corridas en el equipo descrito en la sección 4.3, analizar los resultados y redactar este informe.

Todas las cifras del informe provienen de las mediciones realizadas en ese equipo y quedan respaldadas por los archivos de `resultados/` y las capturas de `evidencias/`; ninguna fue estimada por la herramienta sin medición, salvo la proyección de la pregunta 5, que se presenta explícitamente como estimación. Los estudiantes revisaron el código, los resultados y las conclusiones, y son responsables del contenido entregado.